Contents

Prompt Library **1**
Part 0 — How to use these . . . 2

Part 1 — Core prompts **2**
1. Session bootstrap . . . 2
2. Plan before code . . . 2
3. API endpoint design . . . 3
4. Concurrency review . . . 3
5. Security review . . . 3
6. Tests . . . 4
7. Draft an ADR . . . 4
8. Record a lesson . . . 4
9. Record a migration . . . 4
10. Update a standard . . . 5
11. Keep the SDD honest . . . 5
12. Review a diff . . . 5
13. Load test preparation . . . 5
14. Defence rehearsal . . . 5
15. Brain health check . . . 5
16. Status query — is it built? . . . 6

Part 2 — Unattended operation **6**
2.1 The loop . . . 6
2.2 Stage 1 — pick the next work item . . . 7
2.3 Stage 2 — write the spec . . . 7
2.4 Stage 3 — implement . . . 8
2.5 Stage 4 — verify and self-heal . . . 8
2.6 Stage 5 — close out . . . 9
2.7 Stop conditions . . . 9
2.8 Guardrails to state in every unattended run . . . 9
2.9 Resuming after a stop . . . 10

Part 3 — Tool-specific **10**
3.1 Claude Code . . . 10
3.2 Gemini CLI . . . 11
3.3 Cursor, Copilot, Aider, Codex . . . 11
3.4 Any agent, from CI . . . 11

Part 4 — Snippets **12**
4.1 Unattended driver, tool-agnostic . . . 12
4.2 Local gate the agent must pass . . . 12
4.3 Result envelope . . . 13
4.4 Spec status sweep . . . 13
4.5 Nightly Brain health check . . . 13

Part 5 — Anti-patterns **13**
Related . . . 13

Prompt Library

Prompts for working on `music-room` with an AI coding agent.

- **Part 1** is agent-agnostic. Every block works in Claude Code, Gemini CLI, Cursor, Copilot, Aider, Codex or a plain API call.
- **Part 2** is unattended operation: spec loop, self-healing build, stop conditions, guardrails.
- **Part 3** is per-tool extras, including Claude Code features that have no equivalent elsewhere.
- **Part 4** is glue: scripts, envelopes, CI hooks.

Turkish reading aid: [[prompt-library-tr]]. **This English file is canonical.**

Part 0 — How to use these

The contract every agent reads first

`AGENTS.md` at the repository root. Claude Code additionally reads `CLAUDE.md`, Gemini CLI reads `GEMINI.md`; both point back to `AGENTS.md`. Keep the shared contract in one place.

That is why the prompts below are short: the rules are already loaded. A prompt that re-explains the project is a prompt that will drift from it.

Portability rules for anything you add here

1. **No tool-specific syntax in Part 1.** No slash commands, no `@file`, no tool names.
2. **Reference files by repository-relative path.** `docs/owasp/TOP10-CHECKLIST.md`, never "the OWASP doc".
3. **Ask for a file list before edits.** Works everywhere, and it is the only reliable way to keep an agent inside scope.
4. **Demand a named artifact**, not a feeling. "Report one line per finding as path:line" beats "review this carefully".
5. **State the stop condition.** Unbounded prompts produce unbounded diffs.

Substitutions

`<...>` is yours to fill. `$AGENT_CMD` in Part 4 is your agent's non-interactive CLI entry point.

Part 1 — Core prompts

1. Session bootstrap

First prompt of any session. Cheap, and it stops a workspace scan.

```
Read brain/MEMORY.md and follow its router. Do not scan the workspace.
Then tell me in under 10 lines: current scope, the decided stack, and the open issues.
```

Narrower — one component

```
Read brain/MEMORY.md, then brain/projects/musicroom-backend.md only.
Summarise what is decided for the backend and what is still open.
Read nothing else yet.
```

2. Plan before code

```
Task: <what you want>.

Before writing any code:
1. Read brain/projects/musicroom-backend.md and the relevant brain/architecture/ note.
```

```
2. Read the applicable standards via brain/standards-router.md.
3. Read docs/api/ENDPOINTS.md for the contract, and docs/owasp/API-TOP10-CHECKLIST.md.
4. List every file you will create or change, with its path under MusicRoom/.

Then stop and wait for my approval. Write no code in this turn.

Constraints:
- All backend code under MusicRoom/; client code under MusicRoomApp/. reference/ is read-only.
- One command or query per use case; handlers stay thin.
- No secret in source; configuration from .env.
- .editorconfig limits: 40-line methods, complexity 10, 5 parameters, 15 methods per type.
```

After approval

```
Implement exactly the plan you listed. Change no file outside that list.
Add the tests the testing standard requires for the layers you touched.
When done, print the list of files changed and the commands to verify.
```

3. API endpoint design

```
Design the REST endpoints for <context>, following
docs/technical-guide/20-api/rest-api-design/STANDARD.md and the existing shape in
docs/api/ENDPOINTS.md.

For each endpoint give: method, path, request body, response body, every status code it can
return, and both authorisation gates (ownership, and visibility/licence).

Flag every endpoint where a hidden resource must return 404 rather than 403.
Write no code.
```

4. Concurrency review

The subject grades race handling in four places. Run this before calling a feature done.

```
Review <file or feature> for race conditions.
Context: brain/architecture/realtime-architecture.md and docs/SDD.md section 12.

Check specifically:
- can two concurrent requests produce two votes from one user on one track?
- can two concurrent reorders leave the playlist order inconsistent?
- can two clients advance the same playback session and skip a track?
- can two devices both end up holding control?
- is each guarantee enforced by a database constraint, or only by an application check?
- does any broadcast happen before its transaction commits?

For each finding, give the exact interleaving that breaks it, then the fix.
Report nothing you cannot describe as a concrete interleaving.
```

5. Security review

```
Security-review <scope> against, in this order:
- docs/owasp/TOP10-CHECKLIST.md
- docs/owasp/API-TOP10-CHECKLIST.md
- docs/owasp/ASVS-CONFORMANCE.md, committed chapters only
- docs/technical-guide/40-security/
```

```
- brain/constraints/subject-requirements.md section V.6

One line per finding: path:line, severity, checklist id (A01 / API1 / ASVS requirement id),
the problem, the fix.

Check specifically: object-level authorisation on every endpoint and every hub group join;
per-field profile filtering; rate limits on credential endpoints; ID token validation including
aud and nonce; BCrypt cost and the 72-byte input limit; anything logged that must never be logged.

For each finding you report but do not fix, output the row to paste into
docs/owasp/THREAT-REGISTER.md.

No praise. Say nothing about code that is fine.
```

6. Tests

```
Write tests for <feature> following docs/technical-guide/60-quality/testing/STANDARD.md.

Layer: <domain unit | application unit | persistence integration | API integration | realtime>.
Use xUnit with NSubstitute. Persistence tests use Testcontainers against real PostgreSQL,
never an in-memory provider.

Name each test Method_Scenario_ExpectedResult.
Include the negative authorisation case.
If this touches a concurrency hotspot, add a test that runs the operation in parallel and asserts
the final database state. A sequential test proves nothing about a race.
```

7. Draft an ADR

You decide. The agent records.

```
We decided: <decision>.

Create brain/decisions/ADR-XXXX-<slug>.md in the shape of the existing ADRs:
Context, Decision, Consequences, Alternatives rejected.
Use the next free number from brain/decisions/decisions-index.md.

Then propagate:
- add the row to decisions-index.md
- update any project card or architecture note this invalidates
- update the affected section of docs/SDD.md, plus its section 16 and 17 tables
- update any standard this changes, bump its version, regenerate its PDF

In Consequences, write the costs as well as the benefits. I have to defend this.
```

8. Record a lesson

```
Add a row to brain/lessons/lessons-index.md: <what we learned>.
One sentence of durable fact plus its origin. No transcript, no log output, no debugging story.
If it is only true this week, do not add it.
```

9. Record a migration

```
I created migration <name>.
Add its row to brain/migrations/migrations-index.md: type, reversible yes/no, and the rollback
```

```
path if it is destructive. Reference:
docs/technical-guide/30-data/database-design-migrations/STANDARD.md
```

10. Update a standard

```
Update docs/technical-guide/<path>/STANDARD.md: <change>.

Rules:
- keep Official requirements and Project policy separate; never move a team decision into the
  official section
- do not duplicate a rule that lives in another standard; cross-reference it
- bump the frontmatter version: patch for wording, minor for a new rule, major for a rule that
  invalidates existing code
- regenerate the PDF with docs/build-pdf.sh
- regenerate docs/technical-guide/manifest.json
```

11. Keep the SDD honest

```
<decision or design change> landed.

Update docs/SDD.md: the affected viewpoint section, the section 16 traceability table, and the
section 17 issue tables if this opened or closed one.

Rules:
- do not restate ADR rationale in the SDD; reference the ADR
- never write "done" in section 16 for anything with no code
- leave UNKNOWN - requires decision where a decision is genuinely missing
- regenerate the PDF with docs/build-pdf.sh
```

12. Review a diff

```
Review the current diff against the standards that apply to the files it touches.
Output one line per finding: path:line: severity: problem. fix.
Order by severity. Skip formatting nits; dotnet format already gates those.
Say nothing about code that is fine.
```

13. Load test preparation

```
Prepare the Gatling simulation for <scenario>, following
docs/technical-guide/60-quality/load-testing/STANDARD.md.

Give the simulation file, the exact command, and the concurrency ramp. Remind me which machine
facts I must record before running: without a declared server specification the report is invalid.
Measure REST throughput and SignalR connection fan-out separately.
```

14. Defence rehearsal

```
I am defending this project. Quiz me on <area>.
Source your questions from brain/constraints/subject-requirements.md and brain/decisions/.
Ask what a peer evaluator would ask, including "why not <rejected alternative>".
Do not give me the answer until I have attempted it.
```

15. Brain health check

```
Audit the knowledge layer without reading source code:
```

```
- brain/ notes that contradict each other
- UNKNOWN markers that a closed ADR has already answered
- wikilinks that do not resolve
- standards listed in brain/standards-router.md that do not exist, or the reverse
- files, functions or flags named in brain/ that no longer exist
- docs/SDD.md section 16 rows claiming more than the code supports

Report only real mismatches. Invent no work.
```

16. Status query — is it built?

Read-only. Run it before prompt 2, so the plan starts from what the code does rather than from what the documents claim.

```
Read brain/MEMORY.md and follow its router. Do not scan the workspace.

Question: is <X> implemented?

Answer from evidence only:
- the types, endpoints, handlers or migrations that implement it, as path:line under
  MusicRoom/ or MusicRoomApp/
- the tests that cover it, as path:line
- the deciding ADR from brain/decisions/decisions-index.md, by id, with its frontmatter status
- what docs/SDD.md section 16, docs/DELIVERY-PLAN.md and docs/api/ENDPOINTS.md claim about it
- one verdict, in the section 16 vocabulary: done / partly implemented / designed / absent /
  claimed-but-absent

"designed" means an Accepted ADR and no code. That is a normal state here, not a gap.
"absent" means no ADR and no code.
"claimed-but-absent" means section 16 says done or partly implemented and the code does not
support it. Quote the row and the code side by side.
Propose no work, write no code, change no file.
```

Narrower — one rule or one gate

```
Does MusicRoom/ enforce <rule>?

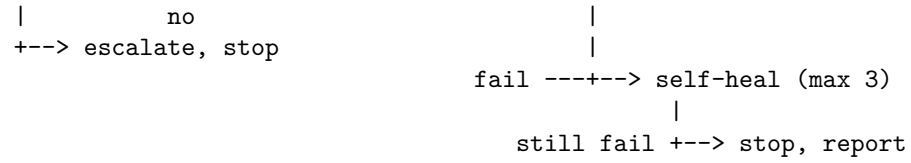
Give three things: the enforcing code as path:line, the test that proves it as path:line, and the
docs/owasp/API-TOP10-CHECKLIST.md row it maps to.
If any of the three is missing, the answer is "not enforced" - name which one is missing.
Write no code.
```

Part 2 — Unattended operation

> **Read this first.** Subject section II is explicit: the team keeps intellectual leadership and must defend every design decision. Autonomy here is therefore bounded to **implementing specs that already sit inside approved decisions**. An agent never chooses architecture, never adds a dependency, and never touches security surfaces without a human. The gate in `docs/specs/README.md` is what makes the difference enforceable rather than hoped-for.

2.1 The loop

```
pick next work  ->  write spec  ->  gate  ->  approved?  ->  implement  ->  verify
```



Every stage is one prompt. Stages are separate so a run can be resumed, inspected and cut short.

2.2 Stage 1 — pick the next work item

Resume before you pick. A blocked spec that a human has since approved is the next unit of work; writing a second spec for the same thing wastes the approval and forks the record.

```
First: list docs/specs/ and read the frontmatter of every SPEC-*.md.

If any spec has status: approved, take the lowest-numbered one, output

  RESUME: <spec path>

and go straight to stage 3. Do not write a new spec. Do not re-run the gate - a human already
answered it, and their answers are in the escalation block. Read them; they are binding.

Otherwise, pick new work:

Read brain/MEMORY.md, then docs/DELIVERY-PLAN.md and docs/SDD.md section 16.

Identify the single next unit of work: the earliest incomplete slice, and within it the smallest
piece that can be completed and verified on its own.

Output only:
  SLICE: <id>
  UNIT: <one line>
  REASON: <one line, why this is next>
  BLOCKED_BY: <issue ids, or none>

If BLOCKED_BY is not "none", stop. Do not choose a different unit to stay busy.
```

2.3 Stage 2 — write the spec

```
Write docs/specs/SPEC-<nnn>-<slug>.md from docs/specs/SPEC-TEMPLATE.md for:
  <UNIT from stage 1>

Fill every section. In particular:
- Files: exact paths, nothing outside them will be touched
- Verification: commands that must pass, and what each one proves
- Gate: answer all seven questions honestly

Before you answer the gate, WALK THE FILES TABLE. Three cycles were lost to tables that looked
complete and were not. For every file you intend to create:

1. Which assembly does it live in, and which assemblies must reference it? Every one of those
   references is a `.csproj` change - list it. A reference that points outward in the dependency
   graph of docs/SDD.md section 7.1 is not a missing row, it is a wrong design: stop and escalate.
2. Does the type cross an assembly boundary? Then it is public and needs its own file, and the
   consumer needs an abstraction it is allowed to see. Name that abstraction's file too.
```

```
3. Does anything actually invoke it? A validator with no ValidationBehaviour, a handler with no
   repository, a registration nothing calls - each is a file that compiles and does nothing.
4. Does it need a CLI, a tool manifest, or a package that is not pinned yet? That is gate 2.
5. Do the four corePackages contain what you assume? They may be empty. Check, do not assume.

Write the answers into the spec. A Files table that has not been walked this way is not finished,
and approving it wastes a human's turn.

If any gate answer is "yes": set status to draft, fill escalation with which gate tripped and
why, and stop. Do not continue.

If all seven are "no": set tier A, status approved, approved_by agent.

Output only the spec path and its status.
```

2.4 Stage 3 — implement

```
Implement docs/specs/SPEC-<nnn>-<slug>.md exactly.

Hard rules:
- change no file outside the spec's Files table
- if you discover the spec is wrong, set its status to blocked, write why in escalation, stop.
  Do not improvise a different design.
- follow AGENTS.md and the standards it points to
- never lower an analyzer severity, delete a test, or skip a test to make something pass

Create branch feat/spec-<nnn>-<slug> from the current branch before the first edit.
Set status to in-progress when you start.
Output the list of files you changed.
```

2.5 Stage 4 — verify and self-heal

```
Run the Verification commands from docs/specs/SPEC-<nnn>-<slug>.md.

If everything passes: complete the spec's "Done when" checklist, set status to done, output
  RESULT: pass
and stop.

If something fails, attempt at most 3 fix cycles. For each cycle:
1. Quote the shortest decisive line of the failure.
2. Classify it: compile error | test assertion | analyzer violation | environment.
3. Fix the cause, inside the spec's Files table only.
4. Re-run the failing command.

Forbidden as "fixes": deleting or skipping a test, weakening an assertion, lowering an
.editorconfig severity, adding a pragma suppression without a justifying comment, widening the
spec's Files table.

If it still fails after 3 cycles, or if the only remaining fix is forbidden:
  set status to blocked
  write escalation with the failure and the fixes you tried
  COMMIT what you have on the spec's branch with a `wip(spec-<nnn>): ...` message, so the
    working tree is clean and the next cycle can resume. Do not push a blocked spec.
  output RESULT: blocked
```

```
  stop.

Committing a block is not claiming success. The evidence is preserved in git instead of in a dirty
tree, and the driver - which refuses to start on a dirty tree - can run again without a human
first tidying up.

Environment failures (Docker not running, no network) are never a code problem. Report them as
RESULT: environment and stop immediately.
```

2.6 Stage 5 — close out

```
For the spec just completed:
- update docs/SDD.md section 16 status honestly (only "done" if code exists)
- add a brain/migrations/migrations-index.md row if a migration was created
- add a docs/owasp/THREAT-REGISTER.md row for any security finding left unfixed
- add a brain/lessons/lessons-index.md row only if something durable was learned
- regenerate docs/api/openapi.json if an endpoint changed
- commit the work on the spec's branch with a Conventional Commits message naming the spec id
- push that branch and open a draft pull request:
  `gh pr create --draft --title "<type>(<scope>): <subject>" --body-file <a file you write>`
  The body states: the spec id, the subject requirements advanced, the verification commands and
  their result, the OWASP checklist outcome, and anything a reviewer must check by hand.
  Do not mark it ready for review. Do not merge.
- output a 5-line summary: unit, files changed, tests added, requirements advanced, what is next
```

2.7 Stop conditions

An unattended run **must** stop at any of these. They are not suggestions; an agent that continues past one is producing an unreviewable diff.

Condition	Why
Any spec gate answers "yes"	Tier B needs a human
3 self-heal cycles exhausted	the cause is not mechanical
A fix would require a forbidden action	the gate exists to prevent exactly this
A file outside the spec's Files table would change	scope creep
A test would be deleted, skipped or weakened	the test is the evidence
Docker or network unavailable	environment, not code
`.agent/STOP` exists	the human pulled the cord
The working tree was dirty at the start	the run would capture unrelated changes
A credential the slice needs does not exist yet	gate 5 escalates; see ADR-0026
Budget exceeded (runs, tokens or wall time)	your loop driver enforces this

2.8 Guardrails to state in every unattended run

Paste this block at the end of any autonomous prompt:

```
Guardrails, absolute:
- Write only under your own application root (MusicRoom/ for backend, MusicRoomApp/ for the
  client), docs/specs/, and the files the spec lists. Never write into the other root.
- reference/ is read-only. Never edit it, never copy from it.
- Never read, print, or commit a secret. Never open .env.
- Never lower an .editorconfig severity, delete a test, or mark a test skipped.
- Never mark a docs/SDD.md section 16 row "done" without code that exists.
- Never write or modify an ADR. Draft one and stop if a decision is needed.
```

```
- Never add or upgrade a dependency.
- Git: create one branch feat/spec-<nnn>-<slug> after approval, and make one commit after
  verification passes. After RESULT: pass only, push that branch and open a DRAFT pull request
  with gh pr create --draft. One pull request per spec.
- Never push to main or to any branch but the spec's own. Never --force, reset --hard, rebase,
  delete a remote branch or tag. Never mark a pull request ready for review, merge it, enable
  auto-merge, or approve it.
- A blocked or environment result pushes nothing.
- Refuse to start on a dirty working tree.
- If .agent/STOP exists, stop immediately and output RESULT: halted.
```

2.9 Resuming after a stop

```
Read docs/specs/ and list every spec whose status is not done, with its status and escalation.
For each blocked or draft spec, tell me in one line what decision or fix it is waiting on.
Change nothing.
```

Part 3 — Tool-specific

3.1 Claude Code

Features with no equivalent in other tools. Use them; do not port them into Part 1.

Feature	Use it for
Plan mode	any change touching more than two files. Present the file list, get approval, then exit plan mode
`/loop <interval> <prompt>`	driving Part 2 stages on a schedule. Omit the interval to let the model self-pace
Subagents	read-only fan-out only: locating code, mapping a directory, surveying naming. Implementation stays in the main thread so one diff is reviewable
`CLAUDE.md`	already points to `AGENTS.md`. Keep it thin; shared rules live in `AGENTS.md`
Slash commands (`.claude/commands/`)	the prompts below, one file each: `/bootstrap`, `/plan`, `/adr`, `/sdd-sync`, `/review-diff`, `/brain-check`, `/status`, `/spec-status`. The library stays canonical
Hooks (`.claude/settings.json`)	mechanical enforcement. Wired: `reference/`, `.env` and every `.pdf` are unwritable — `.claude/hooks/write-boundary.sh`
`/code-review`	the built-in review; still feed it the standards with prompt 12

Autonomous run, Claude Code

From a terminal, not from inside a session:

```
make agent-run            # scoped permissions + live output
make agent-run CYCLES=5
```

A bare `claude -p` **cannot ask for permission**, so every ungranted write is refused and the cycle ends with nothing on disk. `.agent/permissions.json` and `--permission-prompts none` are what make an

unattended run actually produce work. Path rules in that file must use `Edit(...)`; `Write(path)` rules are silently ignored.

From inside an interactive session instead:

```
/loop Read AGENTS.md. Run one full cycle of docs/prompt-library/PROMPTS.md Part 2:
stage 1 pick, stage 2 spec, stage 3 implement, stage 4 verify and self-heal, stage 5 close out.
Apply the guardrail block in 2.8. Honour every stop condition in 2.7.
If .agent/STOP exists, stop the loop immediately.
```

A hook that enforces the write boundary

`.claude/settings.json` — mechanical, so it survives a prompt the model misreads:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "jq -r '.tool_input.file_path' | grep -qE '^(.*/)?(reference)/' && { echo 'refere
          }
        ]
      }
    ]
  }
}
```

3.2 Gemini CLI

- `GEMINI.md` is read automatically; it points to `AGENTS.md`.
- Gemini's large context window invites breadth-first reading. Resist it — the Brain exists so four small files replace a scan. Say "do not scan the workspace" explicitly in the first prompt.
- Non-interactive runs take the prompt on stdin, which is what the Part 4 driver uses.
- No native loop. Use the Part 4 driver script.

3.3 Cursor, Copilot, Aider, Codex

- Cursor: mirror the non-negotiables into `.cursor/rules/` as a short always-on rule that says "read `AGENTS.md`". Do not duplicate the content.
- Copilot: it will not read `AGENTS.md` unaided. Paste prompt 1 at the start of a chat session.
- Aider: `--read AGENTS.md brain/MEMORY.md` on the command line; it respects file scoping well, which suits Part 2 stage 3.
- Any tool without file-scoping: use prompt 2 and check the diff yourself before accepting.

3.4 Any agent, from CI

Keep CI agents **read-only**. A pipeline that edits code turns a red build into a moving target. Good uses: prompt 12 review output as a PR comment, prompt 15 Brain health check on a schedule.

Part 4 — Snippets

4.1 Unattended driver, tool-agnostic

`.agent/run.sh` — works with any agent exposing a non-interactive CLI.

```
#!/usr/bin/env bash
# Drive Part 2 cycles until done, blocked, or the budget runs out.
# Usage: AGENT_CMD="claude -p" ./.agent/run.sh 5
set -euo pipefail
cd "$(git rev-parse --show-toplevel)"

MAX_CYCLES="${1:-5}"
: "${AGENT_CMD:?set AGENT_CMD, e.g. 'claude -p' or 'gemini -p'}"
mkdir -p .agent/runs

for i in $(seq 1 "$MAX_CYCLES"); do
  [ -f .agent/STOP ] && { echo "STOP file present, halting"; exit 0; }

  log=".agent/runs/$(date +%Y%m%d-%H%M%S)-cycle$i.log"
  echo "== cycle $i -> $log"

  $AGENT_CMD "$(cat <<'PROMPT'
Read AGENTS.md. Run one full cycle of docs/prompt-library/PROMPTS.md Part 2:
stage 1 pick, stage 2 spec, stage 3 implement, stage 4 verify and self-heal, stage 5 close out.
Apply the guardrail block in section 2.8. Honour every stop condition in section 2.7.
End your output with exactly one line: RESULT: pass | blocked | environment | halted | nothing-to-do
PROMPT
)" 2>&1 | tee "$log"

  result=$(grep -oE '^RESULT: [a-z-]+' "$log" | tail -1 | cut -d' ' -f2 || echo unknown)
  echo "-- result: $result"
  case "$result" in
    pass)            git --no-pager diff --stat ;;
    blocked|environment|halted|nothing-to-do) echo "stopping: $result"; exit 0 ;;
    *)               echo "unrecognised result, stopping for safety"; exit 1 ;;
  esac
done
echo "budget of $MAX_CYCLES cycles reached"
```

Pull the cord at any time:

```
touch .agent/STOP      # stop after the current cycle
rm .agent/STOP         # resume
```

4.2 Local gate the agent must pass

`make agent-check` — the same gate CI runs, so a self-heal loop converges on the real target.

```
agent-check:
    dotnet format --verify-no-changes
    dotnet build -warnaserror
    dotnet test
    dotnet list package --vulnerable --include-transitive
```

4.3 Result envelope

Any prompt that a script consumes must end with one machine-readable line:

```
RESULT: pass | blocked | environment | halted | nothing-to-do
```

Nothing after it. A driver that parses prose is a driver that misfires.

4.4 Spec status sweep

```
grep -l '^status:' docs/specs/SPEC-*.md 2>/dev/null \
  | xargs -r awk '/^id:|^status:|^escalation:/{printf "%s ", $0} FNR==1{printf "\n%s: ", FILENAME}'
```

4.5 Nightly Brain health check

Read-only, safe to schedule:

```
AGENT_CMD="claude -p" $AGENT_CMD "$(sed -n '/## 15. Brain health check/,/^---/p' \
  docs/prompt-library/PROMPTS.md)"
```

Part 5 — Anti-patterns

Do not	Why	Instead
"Scan the repo and implement voting"	re-reads everything, ignores decisions already made	prompt 1, then 2
"Copy the pattern from reference/nArchitecture"	read-only, net6, and copying breaks subject IV.1	prompt 2 — read the pattern, write your own
"Just make the tests pass"	produces tests shaped around the bug	prompt 6
"Design the architecture for me"	subject section II: you cannot defend what you did not decide	prompt 7 — you decide, the agent records
Paste a connection string or key into a prompt	the value is burned the moment it leaves the machine	reference the `.env` key name
"Write an ADR for whatever seems best"	an undefended decision is worse than an open one	decide, then prompt 7
Unbounded autonomous loop	unreviewable diff, exhausted budget	Part 2 stop conditions and the driver's cycle cap
Agent self-approves a Tier B spec	the gate exists because these changes are expensive to undo	escalate; the seven questions are not advisory
Loop that edits code from CI	a red build becomes a moving target	keep CI agents read-only
"Is done?" answered from the plan	docs/SDD.md section 16 drifts ahead of the code	prompt 16 — a verdict must cite path:line

---

Related

[[prompt-library-tr]] · `../../AGENTS.md` · `../specs/README.md` · [[user-manual]] · [[sdd]] · [[delivery-plan]] · [[owasp-index]] · [[technical-guide-index]] · [[MEMORY]]